SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	SYED MUHAFIZ A	**Reg. No.:**	192472282
Section / Batch:		**Date:**	

Code of Conduct

I SYED MUHAFIZ A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar (CFG) Rule Validation and Parse Tree Construction

Grammar used: S → NP VP | NP → Det N | VP → V NP | Det → the | a | N → student | teacher | book | V → reads | likes

Approach: A recursive-descent parser is written. Each grammar symbol (S, NP, VP, Det, N, V) is tried against the input tokens starting from a position. If a non-terminal has multiple productions, each is attempted until one succeeds. A TreeNode class stores the parse tree, which is printed in an indented form when a sentence is valid. If no production consumes the sentence completely, "Invalid Sentence" is returned.

Python Code:

```
# Experiment 1: CFG Validation and Parse Tree Construction

class TreeNode:
    def __init__(self, label, children=None):
        self.label = label
        self.children = children if children else []

    def __str__(self, level=0):
        ret = "  " * level + self.label + "\n"
        for child in self.children:
            if isinstance(child, TreeNode):
                ret += child.__str__(level + 1)
            else:
                ret += "  " * (level + 1) + child + "\n"
        return ret

# Grammar definition
grammar = {
    'S': [['NP', 'VP']],
    'NP': [['Det', 'N']],
    'VP': [['V', 'NP']],
    'Det': [['the'], ['a']],
    'N': [['student'], ['teacher'], ['book']],
    'V': [['reads'], ['likes']]
}

def match(symbol, tokens, pos):
    """Try to match a symbol (terminal or non-terminal) starting at pos."""
    if symbol not in grammar:  # terminal
        if pos < len(tokens) and tokens[pos] == symbol:
            return TreeNode(symbol), pos + 1
        return None, pos

    for production in grammar[symbol]:
        children = []
        cur_pos = pos
        success = True
        for sym in production:
            node, new_pos = match(sym, tokens, cur_pos)
            if node is None:
                success = False
                break
            children.append(node)
            cur_pos = new_pos
        if success:
            return TreeNode(symbol, children), cur_pos
    return None, pos


def parse_cfg(sentence):
```

```
    tokens = sentence.strip().lower().split()
    tree, end_pos = match('S', tokens, 0)
    if tree is not None and end_pos == len(tokens):
        return tree
    return "Invalid Sentence"


if __name__ == "__main__":
    test_cases = [
        "the student reads a book",
        "a teacher likes the book",
        "student reads book",
        "the book likes a teacher",
        "reads the student book"
    ]

    for sentence in test_cases:
        print(f"Input: {sentence}")
        result = parse_cfg(sentence)
        if isinstance(result, TreeNode):
            print("Output: Valid Parse Tree")
            print(result)
        else:
            print(f"Output: {result}")
        print("-" * 50)
```

Output (all test cases):

```
Input: the student reads a book
Output: Valid Parse Tree
S
  NP
    Det
      the
    N
      student
  VP
    V
      reads
    NP
      Det
        a
      N
        book

--------------------------------------------------
Input: a teacher likes the book
Output: Valid Parse Tree
S
  NP
    Det
      a
    N
      teacher
  VP
    V
      likes
    NP
      Det
        the
      N
        book

--------------------------------------------------
Input: student reads book
Output: Invalid Sentence
--------------------------------------------------
```

```
Input: the book likes a teacher
Output: Valid Parse Tree
S
  NP
    Det
      the
    N
      book
  VP
    V
      likes
    NP
      Det
        a
      N
        teacher

--------------------------------------------------
Input: reads the student book
Output: Invalid Sentence
--------------------------------------------------
```

Test Case Verification:

Input	Expected Output	Program Output	Match
the student reads a book	Valid Parse Tree	Valid Parse Tree	✓
a teacher likes the book	Valid Parse Tree	Valid Parse Tree	✓
student reads book	Invalid Sentence	Invalid Sentence	✓
the book likes a teacher	Valid Parse Tree	Valid Parse Tree	✓
reads the student book	Invalid Sentence	Invalid Sentence	✓

Experiment 2: Agreement and Feature Structure Checking

Feature rules used: he/she/it → singular, 3rd person; they → plural. runs/writes → singular verb; run/write → plural verb.

Approach: Two dictionaries store feature structures — one for subjects (pronoun → {number, person}) and one for verbs (verb form → {number}). check_agreement(subject, verb) looks up both feature structures and compares the 'number' feature. If both agree in number, it returns True; otherwise False. Unknown words also return False.

Python Code:

```
# Experiment 2: Agreement and Feature Structure Checking

subject_features = {
    'he':   {'number': 'singular', 'person': 3},
    'she':  {'number': 'singular', 'person': 3},
    'it':   {'number': 'singular', 'person': 3},
    'they': {'number': 'plural',   'person': 3},
}

verb_features = {
    'runs':   {'number': 'singular'},
    'writes': {'number': 'singular'},
    'run':    {'number': 'plural'},
    'write':  {'number': 'plural'},
}

def check_agreement(subject, verb):
    subject = subject.strip().lower()
    verb = verb.strip().lower()
```

```
    if subject not in subject_features or verb not in verb_features:
        return False

    subj_feat = subject_features[subject]
    verb_feat = verb_features[verb]

    return subj_feat['number'] == verb_feat['number']


if __name__ == "__main__":
    test_cases = [
        ("he", "runs"),
        ("he", "run"),
        ("they", "run"),
        ("they", "writes"),
        ("she", "writes"),
    ]

    for subject, verb in test_cases:
        result = check_agreement(subject, verb)
        print(f"Subject: {subject:6s} Verb: {verb:6s} -> Agreement: {result}")
```

Output (all test cases):

```
Subject: he     Verb: runs   -> Agreement: True
Subject: he     Verb: run    -> Agreement: False
Subject: they   Verb: run    -> Agreement: True
Subject: they   Verb: writes -> Agreement: False
Subject: she    Verb: writes -> Agreement: True
```

Test Case Verification:

Subject	Verb	Expected	Program Output	Match
he	runs	True	True	✓
he	run	False	False	✓
they	run	True	True	✓
they	writes	False	False	✓
she	writes	True	True	✓

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Grammar used: S → NP VP [1.0]; NP → John [0.6] | Mary [0.4]; VP → runs [0.5] | walks [0.5].

Approach: The sentence is split into exactly two tokens (subject and verb). If both tokens exist in the NP and VP probability tables respectively, the sentence probability is computed as P(S→NP VP) × P(NP) × P(VP). If either token is not covered by the grammar (e.g., 'Peter'), the function returns 0.

Python Code:

```
# Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

# PCFG rules with probabilities
S_rule = {'NP VP': 1.0}

NP_rules = {
    'John': 0.6,
    'Mary': 0.4
}
```

```
VP_rules = {
    'runs': 0.5,
    'walks': 0.5
}

def sentence_probability(sentence):
    tokens = sentence.strip().split()
    if len(tokens) != 2:
        return 0.0

    np_word, vp_word = tokens[0], tokens[1]

    if np_word not in NP_rules or vp_word not in VP_rules:
        return 0.0

    p_s = S_rule['NP VP']
    p_np = NP_rules[np_word]
    p_vp = VP_rules[vp_word]

    return round(p_s * p_np * p_vp, 2)


if __name__ == "__main__":
    test_cases = [
        "John runs",
        "John walks",
        "Mary runs",
        "Mary walks",
        "Peter runs"
    ]

    for sentence in test_cases:
        prob = sentence_probability(sentence)
        print(f"Input: {sentence:15s} -> Sentence Probability: {prob:.2f}")
```

Output (all test cases):

```
Input: John runs       -> Sentence Probability: 0.30
Input: John walks      -> Sentence Probability: 0.30
Input: Mary runs       -> Sentence Probability: 0.20
Input: Mary walks      -> Sentence Probability: 0.20
Input: Peter runs      -> Sentence Probability: 0.00
```

Test Case Verification:

Input Sentence	Expected Probability	Program Output	Match
John runs	0.30	0.30	✓
John walks	0.30	0.30	✓
Mary runs	0.20	0.20	✓
Mary walks	0.20	0.20	✓
Peter runs	0.00	0.00	✓

Summary

All three experiments were implemented in pure Python (no external NLP libraries required), executed successfully, and every test case output matches the expected result given in the assessment sheet. These scripts can be combined into a single Google Colab notebook (one cell per experiment, or one function per cell) named as RegNo_Name_CO3-AT1.ipynb, per the submission instructions.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____